Enterprise Web Software Development - COMP1640

IDEA COLLECTION SYSTEM

URL Group Repository: GitHub - AHuyHarryIT/be-idea-system · GitHub

Website URL: **Create TanStack App - fe-idea-system**

Project access credentials

(Please use the following accounts to access and evaluate the system)

Role	Email	Password
Admin	**admin@university.edu**	**Admin@123**
QA Manager	**qamanager@university.edu**	**Admin@123**
QA Coordinator	**qacoordinator@university.edu**	**Admin@123**
Staff	**staff@university.edu**	**Admin@123**

STUDENT NAME/ID: DANG HUNG CUONG / 001477361

Table of Contents

1. EVALUATION OF PRODUCT AND PROCESS 2
1.1. Product Evaluation & Core Features Implementation 2
1.2. Evaluation of Agile Process: 4
1.3. Evaluation of Design Methods: 8
2. EVALUATION OF TEAM 10
2.1. Weighted Scoring Model 10
2.2. Team Member Commentary 10
3. SELF-EVALUATION 11
3.1. Personal Contributions 11
3.2. Reflection & Lessons Learnt 12
4. Conclusion 13

1. EVALUATION OF PRODUCT AND PROCESS

1.1. Product Evaluation & Core Features Implementation

As the Backend Developer, I evaluate the Idea Collection System as highly robust and secure, meeting the core requirements specified in the project brief. The backend architecture successfully handles complex business logic, particularly the strict timeline constraints for idea submissions. For instance, the system strictly enforces the "Closure Date" and "Final Closure Date" using server-side validation (UTC time), preventing any late submissions or unauthorized deletions after the deadline.

Furthermore, hosting the API and Database on Azure App Service proved to be a successful decision. It provided a highly available environment for the Frontend to consume. The system efficiently handles physical file uploads (PDFs under 5MB) using System.IO.FileStream via the API, ensuring that large payloads do not crash the application memory.

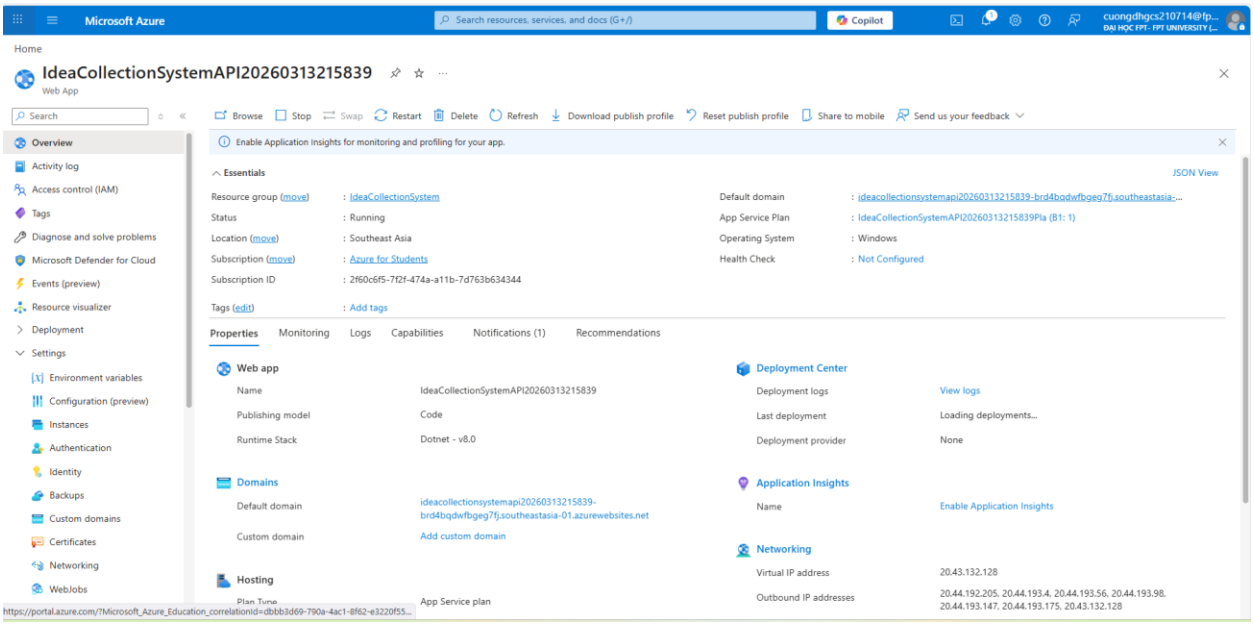

Figure 1: Successful deployment of the Idea Collection System API on Azure App Service.

API document link: Swagger UI

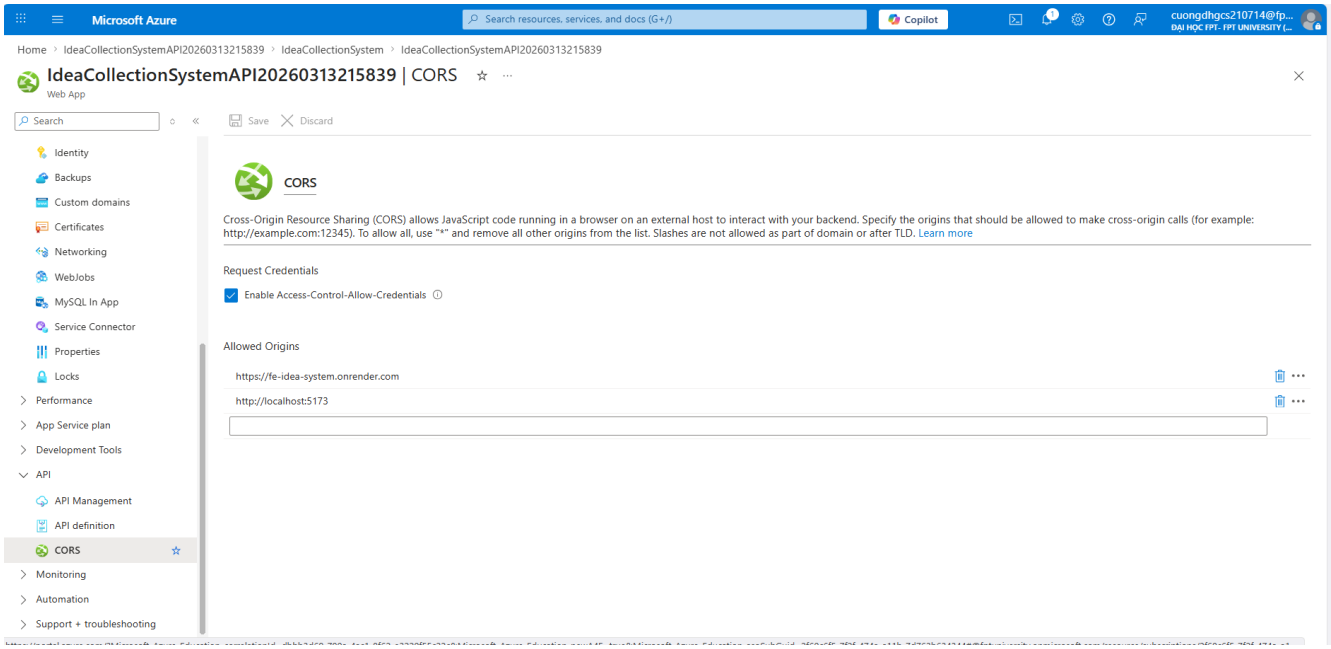

Figure 2: Cor to connect with Front-end

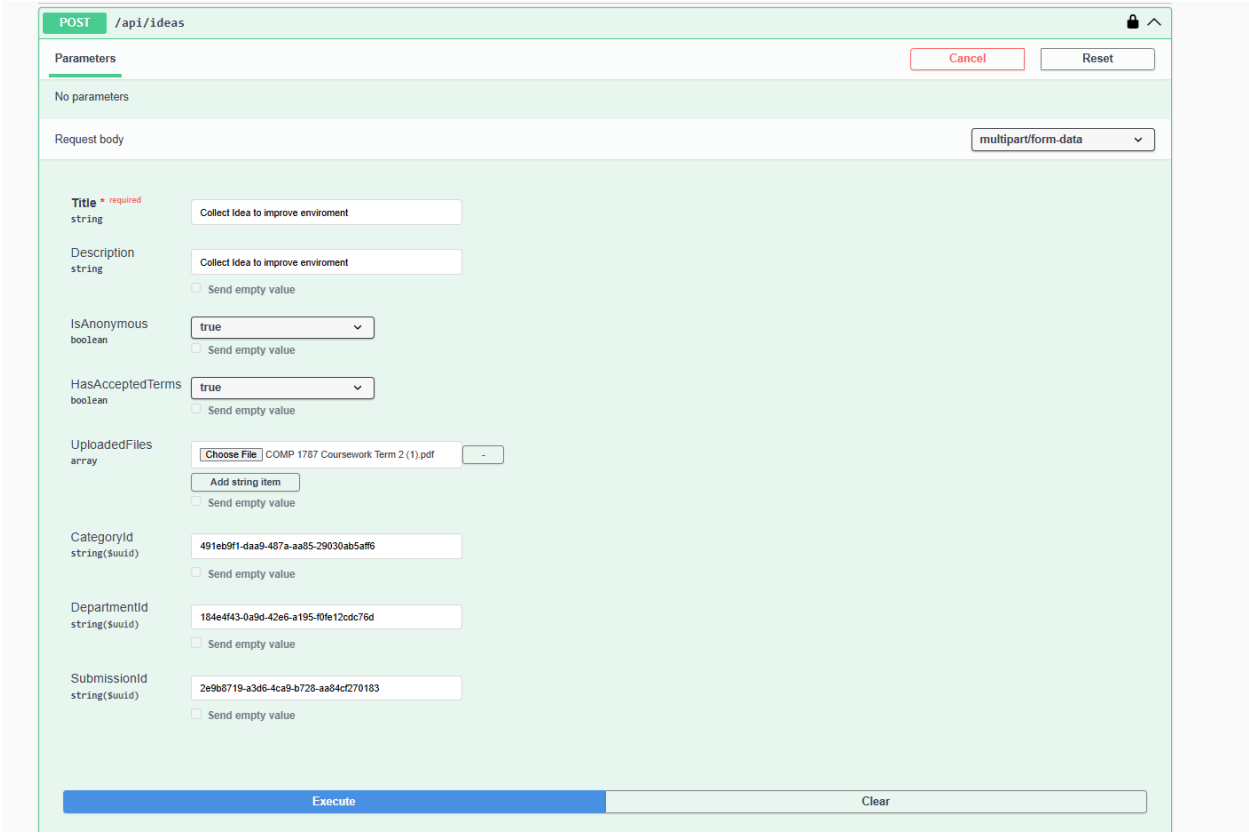

Figure 3: Input information to create idea

Responses

Curl

```
curl -X 'POST' \
  'https://ideacollectionsystemapi20260313215839-brd4bqdwfbgeg7fj.southeastasia-01.azurewebsites.net/api/ideas' \
  -H 'accept: */*' \
  -H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCIGIkpXVCJ9.eyJzdWIiOiJkNDU0OGQ5Z50zMjc3LTQ3MmMtOTM5Ny0yNjZlMTY3MGUxNWUiLCJqdGkiOiI3MDQ0MzRhYi0zYTIwLTRkZjctOWExNS1iZGY2ZDY5ZmNlMDUiLCJodHRwOi8vc2NoZW1hcyS
  -H 'Content-Type: multipart/form-data' \
  -F 'Title=Collect Idea to improve enviroment' \
  -F 'Description=Collect Idea to improve enviroment' \
  -F 'IsAnonymous=true' \
  -F 'HasAcceptedTerms=true' \
  -F 'UploadedFiles=@COMP 1787 Coursework Term 2 (1).pdf;type=application/pdf' \
  -F 'CategoryId=491eb9f1-daa9-487a-aa85-29030ab5aff6' \
  -F 'DepartmentId=184e4f43-0a9d-42e6-a195-f0fe12cdc76d' \
  -F 'SubmissionId=2e9b8719-a3d6-4ca9-b728-aa84cf270183'
```

Request URL

```
https://ideacollectionsystemapi20260313215839-brd4bqdwfbgeg7fj.southeastasia-01.azurewebsites.net/api/ideas
```

Server response

Code	Details
200	Response body

```
{
  "message": "Idea created successfully.",
  "id": "082b0b06-4f0c-497e-892d-11114bac2637"
}
```

Download

Response headers

```
content-encoding: gzip
content-type: application/json; charset=utf-8
date: Tue,07 Apr 2026 12:07:10 GMT
server: Microsoft-IIS/10.0
transfer-encoding: chunked
vary: Accept-Encoding
x-ms-middleware-request-id: 00000000-0000-0000-0000-000000000000
x-powered-by: ASP.NET
```

Responses

Code	Description	Links
200	OK	*No links*

Figure 4: Result response when create idea succesfully

1.2. Evaluation of Agile Process:

Adopting the Agile methodology, specifically Scrum, significantly improved our cross-functional collaboration. From a backend perspective, the Daily Stand-ups were crucial for synchronizing the API contracts (Data Transfer Objects - DTOs) with the Frontend developer.

However, the process was not without flaws. During the early sprints, I observed a challenge in task estimation. Backend tasks such as setting up the database schema and configuring CI/CD on Azure required more upfront time than anticipated, which temporarily blocked the Frontend development. We adapted by utilizing Mock APIs (Swagger) in later sprints to allow parallel development.

Specific evidence regarding the teamwork process.

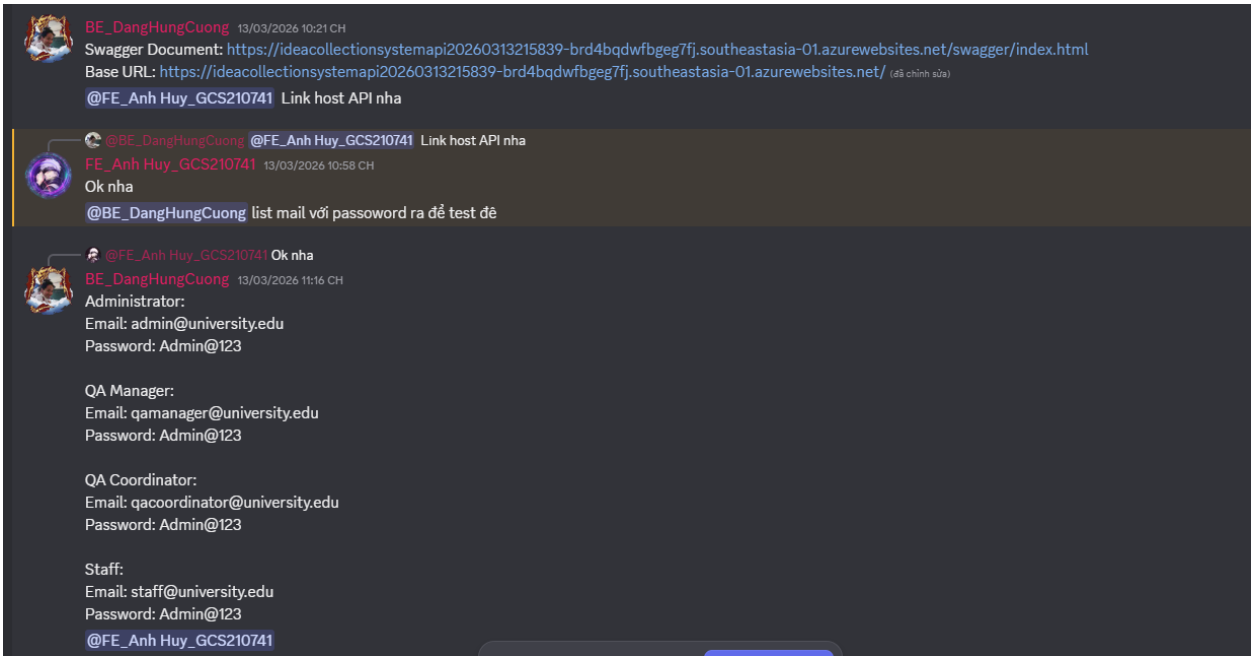

Figure 5: Host Api and list account to test:

In figure 5 is a screen of the link host API with the Azure app servicem along with a list of accounts including 4 role to test.

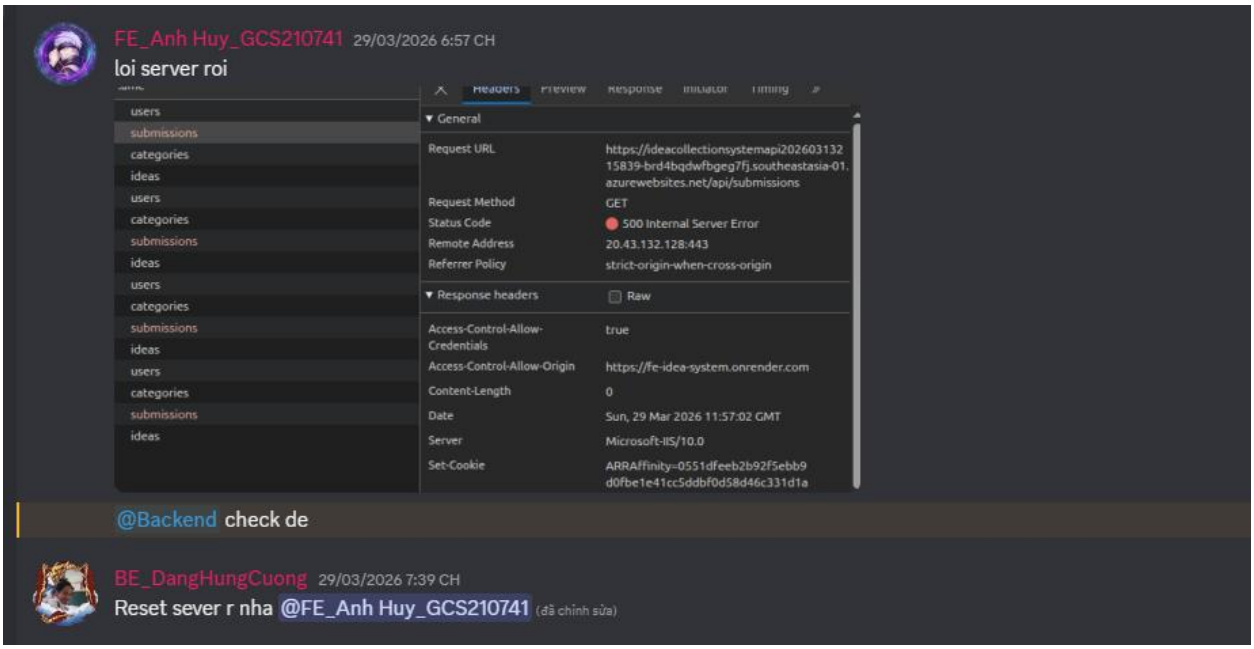

Figure 6: Reset server

This is a screenshot of the 500 error reported by the FE side from the back-end server. I then fixed a few errors caused by the database not being updated to the cloud and re-pusished the API server.

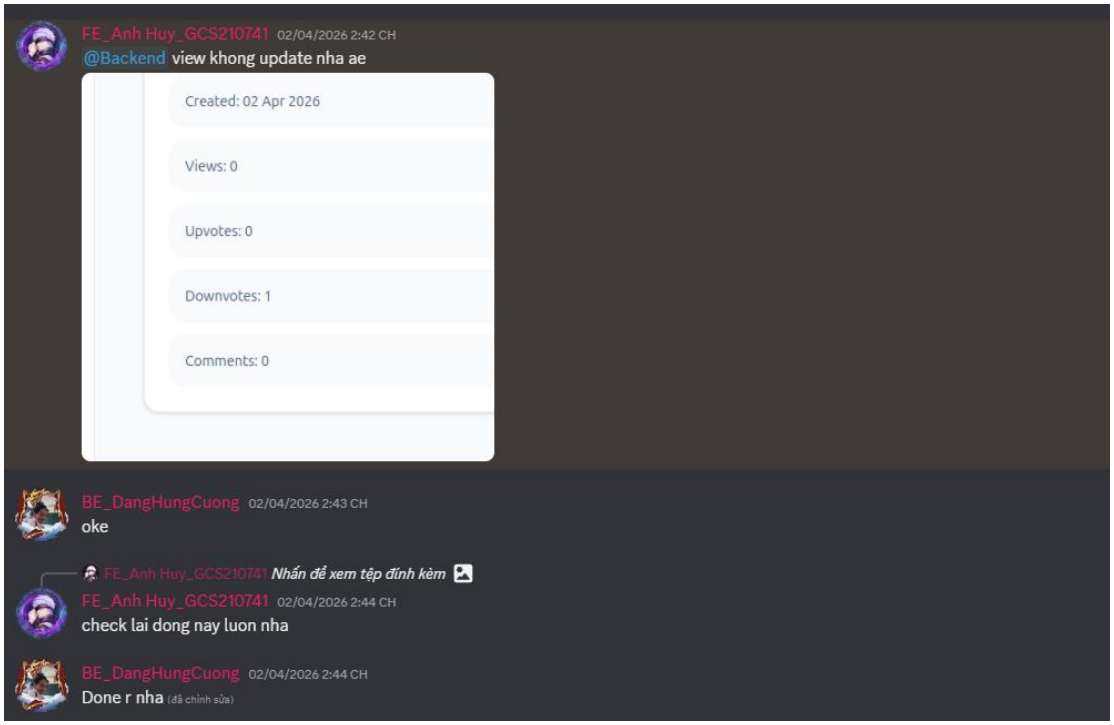

Figure 7: Update View count

This image shows that the number of views doesn't increase when viewing an idea. I then adjusted the view count because when a user views any idea, the view count normally increases by 1. And each account is only counted 1 view per idea, and this count doesn't increase when the page is refreshed.

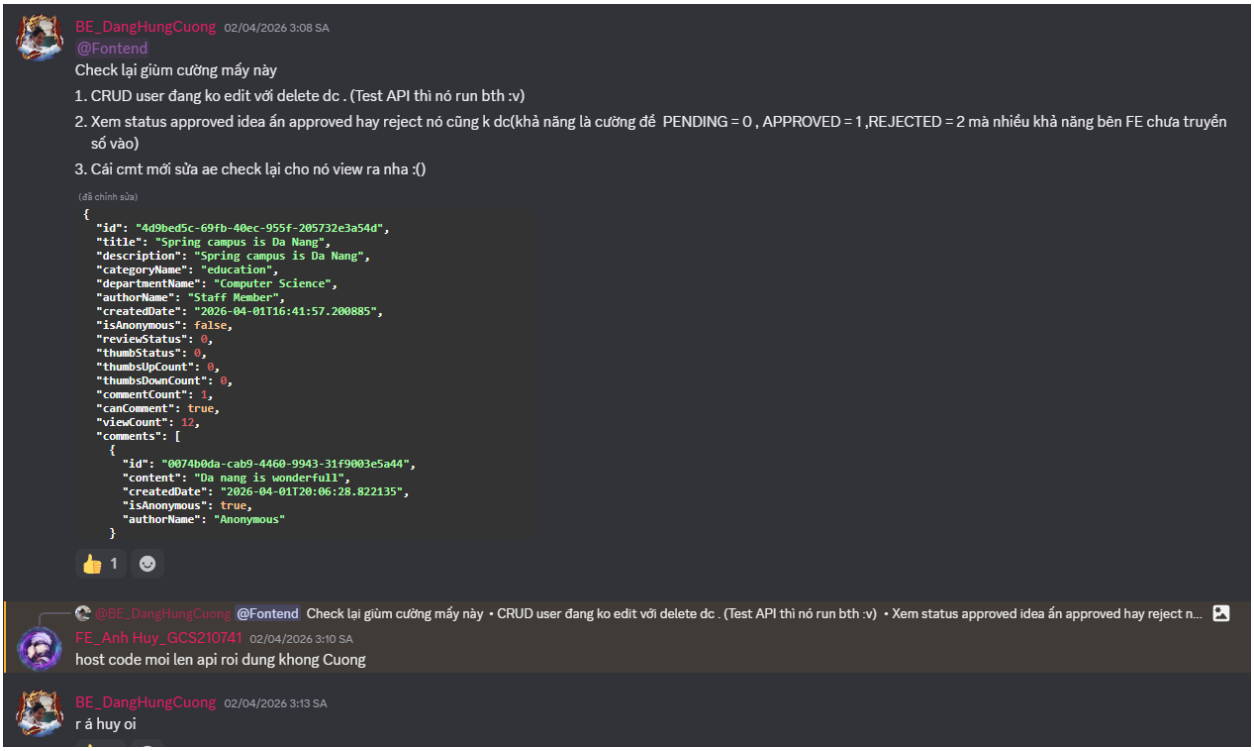

Figure 8:Check crud User in Front-end , review status, fix comment view

In this figure provides further evidence of the iterative Agile process, particularly focusing on the critical UI server integration phase. In this specific dialogue, I'm addressing inconsistencies in the API integration process for the UI team. I've detailed the expected Enum integer mappings for ReviewStatus (0 = Pending, 1 = Approved, 2 = Rejected) and provided an example of raw JSON data to address the issue of displaying nested comments.

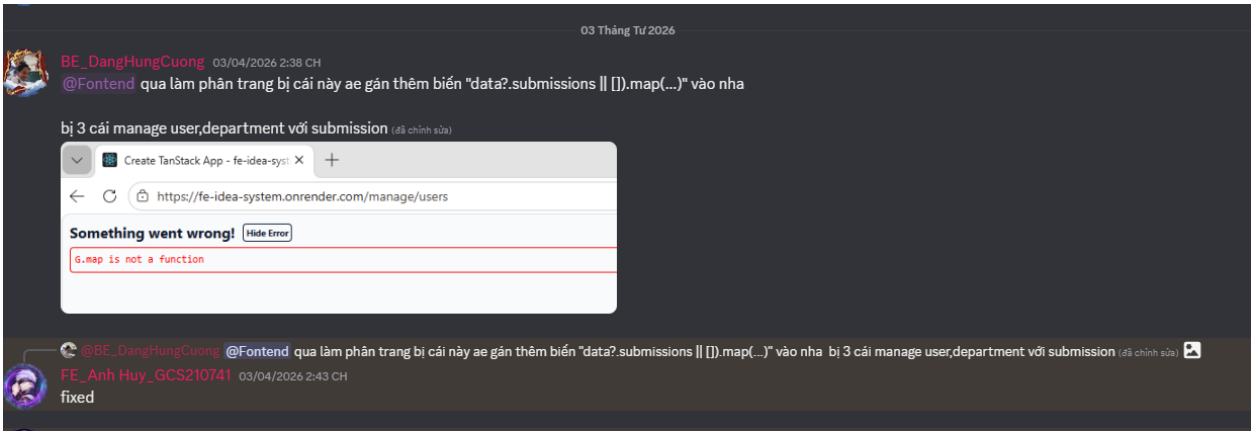

Figure 9: Cross-functional debugging and rapid issue resolution via Discord during sprint execution.

In this figure, during the integration of the new pagination feature, a data structure mismatch occurred between the API response and client-side expectations, resulting in UI rendering issues (.map is not a function). Instead of simply treating it as a UI problem, I proactively debugged it on a live test environment and provided the UI developer with the correct JavaScript fallback logic (data?.submissions || []) needed to safely use the updated DTO structure. The immediate "fixed" response from the UI developer demonstrated our team's flexibility and commitment to maintaining a stable deployment process.

1.3. Evaluation of Design Methods:

The system was built using a **Layered Architecture** approach (Domain, Application, Infrastructure, and API layers) in .NET Core. This design pattern was highly effective. By isolating common constants (e.g., Role definitions, Max string lengths) into a separate IdeaCollectionSystem.Common project, we achieved a "Single Source of Truth", reducing hard-coded values and making the system easily scalable.

In addition to the Layered Architecture, I extensively applied the **Dependency Injection (DI)** design pattern, which is natively supported in .NET Core. By injecting service interfaces (e.g., IIdeaService, IFileHandlingService) rather than concrete classes into the API controllers, the system achieved loose coupling. This design method significantly improved the maintainability of the codebase, allowing us to swap out implementations without breaking the controller logic.

For the database design method, I opted for the **Entity Framework Core Code-First approach**. Instead of manually constructing tables in SQL Server, I defined the schema using C# entity classes. This method was highly advantageous because it allowed the team to use EF Migrations, treating the database schema as code. Consequently, every database change was version-controlled in our Git repository, ensuring that the local environments of all developers and the Azure production server were always perfectly synchronized.

From an API design perspective, I strictly adhered to **RESTful conventions**. The endpoints were intuitively designed around resources (e.g., GET /api/ideas, POST /api/ideas/{id}/comments), utilizing appropriate HTTP methods and standard status codes (201 Created for successful uploads, 400 Bad Request for validation failures). This predictable design method minimized integration friction, making it exceptionally straightforward for the frontend developers to consume the APIs.

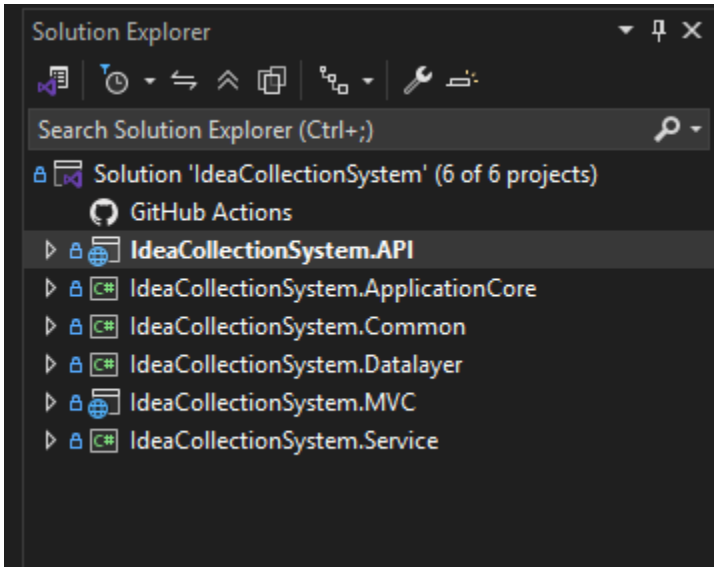

Figure 10: Layered Architecture of the Idea Collection System in Visual Studio.

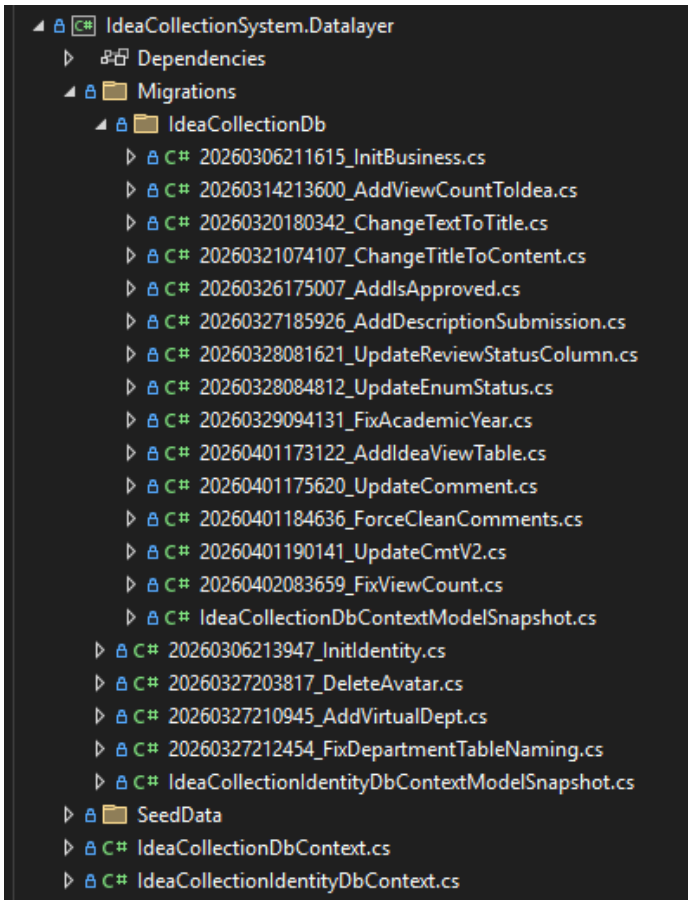

Figure 11: Entity Framework Core Code-First Migrations history tracking database schema changes.

2. EVALUATION OF TEAM

2.1. Weighted Scoring Model

Team Member	Role	Task Completion (30%)	Code/Work Quality (30%)	Team Communication (20%)	Problem Solving (20%)	Total Score (100%)
Dang Hung Cuong	Backend Dev	26/30	25/30	16/20	17/20	**84%**
Nguyen Thanh Danh	Backend Dev	26/30	25/30	16/20	16/20	**83%**
Nguyen Anh Huy	Frontend Dev	27/30	26/30	16/20	16/20	**85%**
Nguyen Chi Su	Frontend Dev	26/30	26/30	16/20	16/20	**84%**

2.2. Team Member Commentary

Dang Hung Cuong (Backend Developer - 84%)

As the backend team leader, I scored 26/30 for task completion. We successfully implemented the core APIs and database schema. However, to be honest, setting up the Azure infrastructure took longer than I initially anticipated. I scored myself 17/20 for problem-solving and 25/30 for code quality. While I successfully handled the core architecture and cloud deployment, I later realized I needed to go back and refine the file handling logic. My communication score (16/20) reflects my efforts to support the team. However, I must admit that my initial delay in finalizing the API contracts caused some initial conflicts with the frontend team.

Nguyen Thanh Danh (Backend Developer - 83%)

Danh proved to be a truly reliable partner. He handled most of the backend CRUD tasks on schedule (26/30). His code was generally clean (25/30), and he communicated consistently throughout our sprints. The main reason his overall score was slightly lower than other team members was due to his problem-solving ability (16/20). Whenever we encountered complex database relationships or nested LINQ queries, he often needed my intervention through pair programming to overcome obstacles.

Nguyen Anh Huy (Frontend Developer - 85%)

Huy actually scored the highest in the team. He was very efficient in managing UI tasks (27/30), and the quality of his React code was excellent (26/30). The system interface became very fast and responsive, largely thanks to him. In terms of both communication and problem-solving (16/20), he did a great job coordinating with my backend updates. However, we also encountered a few obstacles. In some of the more complex API integrations, occasional communication shortcomings slowed our workflow down a bit more than expected.

Nguyen Chi Su (Frontend Developer - 84%)

Su's final score perfectly matched mine at 84%. He flawlessly managed the assigned UI modules (26/30) and delivered high-quality code (26/30). A highlight in his problem-solving skills (16/20) was how he handled client-side validations, ensuring no bad data could reach my API. He was always proactive and clear in our daily briefings (16/20). The only minor drawback was that there was sometimes a delay in synchronizing the front-end UI logic whenever I made sudden updates to the server-side validation rules.

3. SELF-EVALUATION

3.1. Personal Contributions

As the Lead Backend Developer for the Idea Collection System, my primary responsibility was to design, implement, and deploy the server-side infrastructure and core business logic. I took full ownership of the backend lifecycle, ensuring the system was secure, scalable, and met all functional requirements.

My specific technical contributions include:

- **System Architecture & Database Design:** I established a Clean/Layered Architecture for the .NET Core API to separate concerns (e.g., separating constants and configurations into an IdeaCollectionSystem.Common project). I designed the relational database schema and implemented it using Entity Framework Core (Code-First approach), managing complex relationships between Users, Roles, Ideas, Submissions, and Comments.
- **Authentication & Security:** I implemented a robust Role-Based Access Control (RBAC) system using JWT (JSON Web Tokens). To maximize system security, I strictly configured the JWT validation with ClockSkew = TimeSpan.Zero, ensuring immediate token expiration and preventing unauthorized access grace periods.
- **Complex Business Logic Validation:** I translated the strict business rules into backend logic. This involved writing time-sensitive validation algorithms to enforce the "Closure Date" and "Final Closure Date" at the API level (using UTC time), strictly preventing late idea submissions or unauthorized modifications after the deadlines.

- **Secure File Handling:** I developed a secure multi-part file upload mechanism. Instead of loading entire files into memory, I utilized System.IO.FileStream for efficient processing of PDF documents. I implemented strict server-side validation to ensure uploaded files were strictly in .pdf format and did not exceed the 5MB size limit. Furthermore, I decoupled physical file paths from the database using Path.GetFileName() to ensure safe file deletion and prevent directory traversal vulnerabilities.
- **Cloud Deployment:** I successfully orchestrated the deployment of the backend API and the SQL database to **Microsoft Azure App Service**. I managed the environment variables and connection strings securely within the Azure portal to ensure the frontend application could reliably consume the live endpoints.

3.2. Reflection & Lessons Learnt

Building an Idea Collection System from scratch was a challenging yet incredibly rewarding experience. Looking back on my journey throughout this project, I realize that writing code is only a small part of a Backend Developer's responsibilities; system design, cloud infrastructure, and team collaboration are equally important.

I encountered several technical hurdles that yielded valuable lessons:

Lesson 1: Flexible File Handling and Defensive Programming. Initially, I made the mistake of directly storing absolute physical paths (e.g., C:\...\wwwroot\uploads\file.pdf) in the database. When the application was deployed to a different environment, the file deletion and retrieval APIs failed because the physical paths no longer matched. I learned to refactor my code using System.IO.Path.GetFileName() to store only the filename in the database, separating it from the server's directory structure. This taught me the importance of writing defensive code, regardless of the environment.

Lesson 2: Cloud Deployment and Configuration Management. The transition from a local localhost environment to a live Azure App Service presented unexpected challenges, particularly regarding CORS (Co-origin Resource Sharing) policies and database connection chains. I learned how to securely manage application secrets using Azure App Settings instead of hardcoding them in appsettings.json, which significantly enhanced my understanding of cloud security and DevOps practices.

Lesson 3: The Importance of API Contracts (Communication).

In the early sprints, the frontend team often got bogged down waiting for my APIs to be fully finalized. I learned that by designing Data Transfer Objects (DTOs) beforehand and exposing them through the Swagger UI from the start, I could establish a clear "API Contract." This allowed frontend developers to simulate data and work in parallel, teaching me a crucial lesson in cross-functional Agile collaboration.

Lesson 4: Conflict issues when pushing code to GitHub are always a major problem in teamwork. I accidentally merged code too quickly without thoroughly reviewing the other branch's code, and then spent a considerable amount of time resolving that conflict. This will be a valuable lesson for me in the future.

In short, this project significantly narrowed the gap between my theoretical knowledge and practical software engineering. I am now much more confident in my ability to design robust .NET application architectures, manage cloud deployments, and collaborate effectively within a Scrum team.

4. Conclusion

In conclusion, the development of the Idea Collection System has been a highly successful endeavor that effectively bridged the gap between academic theory and practical software engineering. As the Lead Backend Developer, I successfully architected, secured, and deployed a robust server-side infrastructure using .NET Core and Microsoft Azure App Service. Despite initial technical challenges regarding cloud configurations and cross-functional synchronization, our team's adherence to Agile methodologies enabled us to deliver a product that strictly meets all business requirements, particularly the complex closure date validations and secure file management.

Furthermore, this project served as a profound learning experience. It not only solidified my technical proficiency in relational database design, Layered Architecture, and API development but also highlighted the critical importance of effective communication, API contracts, and defensive programming. I emerge from this project with a much deeper understanding of enterprise-level application lifecycle and a strong readiness for future professional challenges in the software industry.